ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Modernizing C++ Legacy Systems with Rust: Strategies for Safe Interoperability and Performance Evaluation in Rust-Based Boost Components

Filipe Daniel Ferreira

Dissertation submitted in partial fulfilment of the requirements for the Master's Degree in Critical Computing Systems Engineering

Supervisor: Luis Miguel Pinho

Porto, September 10, 2026

Statement of Integrity

I hereby declare having conducted this academic work with integrity. I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

I declare that the work presented in this document is original and my own, and has not previously been used for any other purpose.

I further declare that I have fully followed the Code of Good Practices and Conduct of the Polytechnic Institute of Porto.

ISEP, Porto, September 10, 2026

Abstract

The growing complexity of safety-critical software systems, particularly in the automotive domain, has intensified concerns regarding memory safety, concurrency correctness, and long-term maintainability of large C++ codebases. Although C++ remains the dominant language in embedded and real-time systems due to its performance and mature ecosystem, its reliance on manual memory management continues to be a major source of vulnerabilities. Rust has emerged as a promising alternative, offering strong compile-time guarantees for memory safety and data-race freedom without introducing garbage collection. However, the adoption of Rust in industrial environments is hindered by the scale of existing C++ systems and their deep dependence on template-heavy frameworks such as the Boost C++ Libraries.

This dissertation investigates strategies for incrementally modernizing legacy C++ systems by integrating Rust in a safe and performant manner. Focusing on Boost-based components commonly used in automotive software architectures, it proposes a systematic methodology for selecting migration candidates, designing bidirectional interoperability interfaces, and re-implementing critical functionality in Rust. The work explores Foreign Function Interfaces (FFI) patterns, evaluates existing interoperability tools, and addresses the architectural challenges arising from the mismatch between C++ templates and Rust's ownership model.

A Rust-based reimplementation of selected Boost components is developed and integrated into a hybrid C++/Rust system. Performance, memory usage, and interoperability overhead are experimentally evaluated to quantify the benefits and costs of this approach. The results demonstrate that incremental migration can significantly improve memory safety while preserving determinism and performance, providing practical guidance for the adoption of Rust in high-assurance and safety-critical software systems.

Keywords: Boost, C++, Rust, interoperability, migration, memory safety

Resumo

A crescente complexidade dos sistemas de software crítico e seguro, particularmente no domínio automóvel, intensificou as preocupações relativas à segurança da memória, à correção da concorrência e à manutenção a longo prazo de grandes bases de código C++. Embora o C++ continue a ser a linguagem dominante em sistemas embebidos e de tempo real devido ao seu desempenho e ecossistema maduro, a sua dependência da gestão manual da memória continua a ser uma importante fonte de vulnerabilidades. O Rust surgiu como uma alternativa promissora, oferecendo fortes garantias em tempo de compilação para a segurança da memória e a ausência de conflitos de dados, sem introduzir a recolha de lixo. No entanto, a adoção do Rust em ambientes industriais é dificultada pela escala dos sistemas C++ existentes e sua profunda dependência de frameworks fortemente baseadas em modelos, como as bibliotecas Boost C++.

Esta dissertação investiga estratégias para modernizar incrementalmente sistemas C++ antigos, integrando Rust de maneira segura e com bom desempenho. Com foco em componentes baseados em Boost comumente usados em arquiteturas de software automotivo, ela propõe uma metodologia sistemática para selecionar candidatos à migração, projetar interfaces de interoperabilidade bidirecionais e reimplementar funcionalidades críticas em Rust. O trabalho explora padrões de FFI, avalia ferramentas de interoperabilidade existentes e aborda os desafios arquitetônicos decorrentes da incompatibilidade entre modelos C++ e o modelo de propriedade do Rust.

Uma reimplementação baseada em Rust de componentes Boost selecionados é desenvolvida e integrada a um sistema híbrido C++/Rust. O desempenho, o uso de memória e a sobrecarga de interoperabilidade são avaliados experimentalmente para quantificar os benefícios e custos dessa abordagem. Os resultados demonstram que a migração incremental pode melhorar significativamente a segurança da memória , preservando o determinismo e o desempenho, fornecendo orientações práticas para a adoção do Rust em sistemas de software de alta garantia e segurança crítica.

Contents

List of Figures

List of Tables

List of Abbreviations

List of Acronyms

ASIL	Automotive Safety Integrity Levels.
CWE	Common Weakness Enumeration.
ECU	Electronic Control Units.
FFI	Foreign Function Interfaces.
QM	Quality Management.
RAII	Resource Acquisition Is Initialization.
SCORE	Safety-Critical Open Research Environment.
SDV	Software Defined Vehicles.
STD	Standard C++ library.
TMP	Template Metaprogramming.
WCET	Worst Case Execution Time.

Chapter 1

Introduction

1.1 High-Level Overview and Motivation

The automotive industry is performing a profound transformation driven by the rise of the Software Defined Vehicles (SDV), autonomous driving technologies and increasingly interconnected Electronic Control Units (ECU) (S&P Global Mobility 2025). With the grouth of these systems in complexity and responsability, the reliability of high-assurance software becomes critical. The systems must operate under strict real-time, reliability and safety requeriments regulated by standards as ISO 26262 (Deloitte 2024). In this environment, memory safety has a special importance, as software faults can directly translate into physical hazards (Bristol 2025).

Historically, C++ has been the dominant language (paired with C) in embedded and automotive development due to its performance, deterministic resource use and mature ecosystem (*ISO 26262-6:2018 Road vehicles — Functional safety — Part 6: Product development at the software level* 2018). However, its dependence on manual memory management continues to be a major source of vulnerabilities such as: buffer overflows, dangling pointes, race conditions and undefined behaviors (The MISRA Consortium 2023). Such issues are incompatible with the needs ofsafety critical cyber-physical systems, where a minor software defect can lead to serious consequences (AUTOSAR Partnership 2023).

Rust has rapidly emerged as a plausible successor for safety-critical software. Enforcing memory safety, ownerhip and data race free concurrency at compile time (*Rust Documentation* 2025). Rust promises to drastically reduce classes of defects edemic to C/C++ systems (Blandy, Orendorff, and Tindall 2021). Rust achieves this without introducing garbage collection, retaining the predictability required for real-time environments (*Rust Documentation* 2025).

Despite the advantages Rust presents, the transition from C++ to Rust in large industrial ecosystems remains complex. Automotive software contains millions of lines of validated legacy C++ code. This code is oftenly built at top of complex frameworks and libraries as Boost C++ libraries (Tolnay 2025). Rewrinting those systems from scratch is infeasible. To

avoid it, the companies usally pursue incremental and hybrid migration. This migration requires robust interoperability between Rust and C++ (Google Android Team 2024).

This dissertation is motivated byt the need to address that gap: enabling safe, practical and measurable integration of Rust into deeply entrenched C++ based automotive architectures, specially real-time related architectures.

Rewriting millions of lines of validated, legacy C++ code is operationally infeasible, necessitating a hybrid integration strategy. This transition is particularly problematic for automotive subsystems (such as sensor fusion, V2X communication and path planning) that are deeply coupled with the C++ ecosystem and reliant on complex, template-heavy frameworks like the Boost C++ Libraries.

1.2 Problem Statement

The central results from the tension between two different realities:

1. **C++ is deeply entrenched in automotive systems**, specially through heavily templated, performance-critical frameworks like Boost.
2. **Rust provides strong memory safety guarantees**, but its adoption is hindered by the difficulty of interoperating with complex C++ abstrations creating a challenge for a transition between C++ and Rust.

Currently, the interoperability approaches rely mainly on C-style Foreign Function Interfaces (FFI) . However, ist not possiblet to natively represent C++ templates, ownership semantics, RAII lifecycles or advanced Boost abstractions with C FFI (Google 2021).

As result of this limitation:

- The integration often requires unsafe code, reducing Rust's safety benefits.
- Interfacing with templaty heavily libraries, as Boost.Geometry and Boost.Asio, introduces high engineering overhead.
- Crossing the Rust and C++ boundary imposes a performance cost that is poorly characterized for high-assurance systems.
- No established methodology exists for the systematic and incremental migration of C++ Boost components to Rust while maintaining compatibility with real-time automotive subsystems.

The problem is not simply porting code from C++ to Rust, but the absence of validated techniques, architectural patterns, and tooling to bridge the impedance mismatch between C++ templates and Rust's ownership model, without breaking determinism, performance or safety requirements.

1.3 Research Questions

To address this gap, this dissertations is guided by the following research questions:

1. How can Rust be integrated into existing C++/Boost based automotive systems in a way that preserves and improves safety, determinism and performance?
2. What are the technical criteria required to identify wich Boost library components are the most viable candidates for a migration to Rust within an automotive context?
3. Which Boost components represent suitable candidates for partial or complete migration to Rust, considering complexity, memory safety risk and template dependency?
4. What architectural patterns and FFI strategies enable safe and efficient bidirectional communication between Rust and C++?
5. What measurable performance and memory safety benefits does a rust based reimplementation provide when compared with its Boost counterpart?
6. What are the interoperability costs ("FFI tax") and how do they impact hybrid real-time systems?

1.4 Research Objectives

The primary objective of this dissertation is to design, implement and evaluate a methodology for the incremental migration of Boost C++ Boost components into Rust, ensuring safe and efficient interoperability within hybrid systems. This involves re-architecting critical components to leverage Rust's memory safety guarantees and evaluating resulting architectural trade-offs.

The specific objectives are

- Analyze the current interoperability mechanisms between Rust and C++ (e.g., CXX, Bindgen, autocxx, cbindgen).
- Identify and define criteria for selecting Boost components suitable for migration (eg. memory-safety relevance, template complexity, use in automotive systems).
- Re-architect and re-implement a selected Boost component in Rust while preserving semantics and meeting strict ownership and borrowing guarantees.
- Design bidirectional Rust and C++ interfaces enabling both C++ to call Rust and Rust to call C++ components.
- Measure runtime performance, memory footprint and the overhead of FFI interactions.
- Evaluate the overall feasibility, benefits and limitations of incremental C++ to Rust migration in high-assurance environments.

1.5 Research Contributions

The contributions of this dissertation are:

- A comprehensive overview of Rust, C++ and Rust interoperability and the Boost library ecosystem.
- A systematic analysis of existing interoperability mechanisms and their suitability for template-heavy automotive codebases.
- A formalized methodology for selecting and migrating Boost components to Rust.
- A Rust-native reimplementation of selected Boost component with full memory safety guarantees.
- A robust bidirectional FFI integration layer
- A Performance evaluation comparing Rust and C++ implementations and quantifiying overhead

1.6 Dissertation Plan Structure

The remainder of this dissertation plan is organized as follows:

- **Chapter 2 - State of the Art:**
  Presents the tehcnological backgrund and related work. Includes Rust fundamentals, C++ interoperability mechanisms, Boost library architecture and prior migration initiatives.
- **Chapter 3 - Methodology and Solution proposal:**
  Defines the criteria for selecting Boost components and describes the proposed migration methodology and architectural designed.
- **Chapter 4 - Development Plan:**
  Details the Rust reimplementation, interface design and bidirectional FFI integration.

1.7 Privacy and ethics

In the context of this dissertation plan, which is focused in modernizing C++ based systems using Rust, ethical considerations were primarily centered on scientific integrity, software safety and intellectual property compliance.

The research relied on public open-source codebases, as Boost libraries. No personal data, user-identifiable information (PII), or private operational logs were collected, processed, or stored. Consequently, the study falls outside the scope of GDPR requirements regarding human subjects.

1.8 Use of AI Tools

For the preparation of this dissertation plan and supporting documentation, AI tools (specifically large language models) were used strictly for the following purposes:

- **Document writing evaluation and improvement**, including linguistic refinement, clarity enhancement, and reorganization of text originally written by the author.
- **Assistance in identifying potential sources and directions for literature review**, such as suggesting relevant topics, keywords, and areas of research to investigate. All cited references included in the dissertation were individually verified, read, and selected by the author.
- Structural feedback and general writing suggestions for non-technical sections.
- Code example generation to support the investigation of the state of the art and also to facilitate the understanding.

The tools used were Perplexity (latest version), for deep search specialy for community based information, chat GPT (version GPT5.1) and Gemini (version 3), for document writing evaluation and improvement.

No AI-generated technical explanations, conceptual developments, design decisions, code, or analysis results were incorporated directly into the dissertation. All technical content is exclusively authored by the candidate.

The author takes full responsibility for the correctness, originality, and academic integrity of the work presented. If additional AI tools are used during the subsequent stages of the dissertation, this section will be updated accordingly to specify which tools were used, for what purpose, and in which components of the document. If no further AI usage occurs, this will also be explicitly stated.

Chapter 2

State of the art

The integration of Rust into C++ environments, particularly Boost libraries, marks a significant evolution in software development. Rust's core strengths are memory safety and concurrency without garbage collection, addressing longstanding vulnerabilities in C++ systems. Recent studies on microcontroller applications show that Rust can have similar speed to C with lower energy use than interpreted languages. While performance evaluations on platforms, such as Raspberry Pi, indicates that Rust FFT implementations can match or exceed C in energy efficiency (Dashdamirli 2025; Rooney and Matthews 2023).

These characteristics are dominant in systems with strict deterministic requirements, such as automotive Electronic Control Units (ECUs), high-frequency trading platforms, and real-time robotics. In these domains, the non-determinism introduced by garbage-collection "stop-the-world" pauses is not appropriate due to hard real-time latency constraints. Furthermore, resource-constrained embedded environments often cannot afford the runtime overhead associated with managed languages. Rust solves this problem through an ownership model and a strict borrow checker enforced at compile time. By tracking the lifetime and scope of every value during compilation, Rust guarantees memory safety and data-race freedom without requiring a runtime garbage collector. (Okawa and Yamaguchi 2024) This ensures that the resulting binary maintains the predictable performance and low resource footprint of C++, while eliminating entire classes of bugs such as null pointer de-referencing and use-after-free errors.

2.1 Rust

2.1.1 Overview

Rust is a systems programming language focused on three goals: safety, speed, and concurrency (Klabnik, Nichols, and Developers 2017).

It is designed to ensure memory safety and high performance without relying on a garbage collector, present in languages as Python or Java.

To not rely on garbage collection and to avoid common memory issues found in languages like C and C++, such as null pointer dereferencing, double frees, and segmentation faults, Rust was designed to be memory safe while offering comparable performance (Khan 2023).

Ownership and Memory Safety

Rust distinguishes itself through a rigorous ownership model, which manages memory through a strict set of rules enforced at compile time by the "borrow checker", without a garbage collector. (Blandy, Orendorff, and Tindall 2021)

The garbage collector, used by Python and Java, is an automatic memory management process that helps programs to run efficiently. Rust does not use it for memory management, instead it determines exactly when memory should be freed based on variable scope. (Blandy, Orendorff, and Tindall 2021)

Within a program run, every value has a single "owner" and when the owner goes out of scope the value is dropped. Ownership can be transferred (moved) but not implicitly copied for complex types, preventing double-free errors. (Blandy, Orendorff, and Tindall 2021)

I also possible to "borrow" values through references. The compiler ensures that references are always valid and a mutable references cannot exist while other references exists, preventing data races. (Khan 2023)

Scope and ownership code example:

```rust
fn main() {
// 1. SCOPE AND OWNERSHIP
{
    let s1 = String::from("hello"); // s1 owns the string
    // s1 is valid here
} // s1 goes out of scope here; memory is automatically freed (dropped)

// 2. MOVE SEMANTICS (Transferring Ownership)
let s2 = String::from("complex type");
let s3 = s2; // Ownership moved from s2 to s3

// println!("{}", s2); // Error! s2 is invalid. Prevents double-free errors.
println!("s3 owns the data: {}", s3);

// 3. BORROWING (References)
let mut s4 = String::from("data");

// Immutable borrow
let r1 = &s4;
let r2 = &s4; // Multiple immutable references are allowed
println!("Read only: {} and {}", r1, r2);
// r1 and r2 scopes end here (last usage)
```

```
23
24      // Mutable borrow
25      let r3 = &mut s4;
26      r3.push_str(" race"); // Modify the borrowed value
27      // let r4 = &s4; // Error! Cannot have immutable ref while mutable
        ref exists
28                          // This rule prevents data races.
29
30      println!("Modified data: {}", r3);
31  }
```

Tooling and ecosystem

An high quality tooling is a strong base of the Rust development environment. The integrated package manager **Cargo**, documentation generator **rustdoc** and built-in testing framework significantly enhance developer productivity. The Rust package registry, crates.io, is an important source of libraries for different applications.

Beside the growing usage of Rust, it still relies on interoperability with C/C++ ecosystem, specially for low-level applications (eg. hardware interactions, graphics, legacy protocols). Interfacing Rust with foreign code is possible using through the FFI. Usualy, an FFI is functional and efficient but often requires carefull design of abstractions, specially when crossing language boundaries with different memory management models.

Adoption and Industrial Applications

Rust has been adopted by several domains, as:

- Web browsers and rendering engines, as mozilla firefox (Mozilla 2026).
- Cloud infrastructures, as AWS firecracker (Amazon Web Services 2026).
- Operating systems and kernel modules, as in Linux kernel (The Linux Kernel Organization 2026).
- Blockchain clients and cryptographic systems (Kuo 2024).
- High performance networking and embedded systems (The Rust Project Developers 2026).

2.2 C++ and the Boost Library

2.2.1 C++

C++ is currently the dominant programming language due to its extensive ecosystem. It has hardware specific libraries, mature compilers, profiling and debugging tools and frameworks for high-performance computing. Is also used for a large group of companies for many years (Stroustrup 2013). It leads to massive codebases, thousands of lines of code that make large rewrites infeasible (CppCon 2020).

Some common industries that usually use C++ are telecommunications, automotive, finance, aerospace, gaming, real-time control and large scale cloud systems (Stroustrup 2020).

The main challenges these systems have is their modernization. Modernizing them while ensuring the continuity, comaptibility and performance is not easy. Due to this, languages that interoperate with C++ instead of replacing it are preferred.

The main strenghts of C++ are:

- High performance (specially comparing with C) (Stroustrup 2020).
- Deterministic resource management through Resource Acquisition Is Initialization (RAII) (Stroustrup 2013).
- Support for control of hardware and memory (Stroustrup 2013).
- String abstraction mechanisms as templates, operator overloading and generic programming (Stroustrup 2013).
- Mature tools, compilers, profiling and debugging ecosystems (CppCon 2020).

The evolution of C++ ISO standard introduced modern features as smart pointers, concurrency primitives, modules and constant evaluation, for safety improvement and expressiveness (*ISO/IEC 14882:2020 Programming languages — C++* 2020).

C++ memory model and safety limitations

Beside all the advantages, C++ has memory and safety limitations. C++ is designed to give developers complete control over memory and resources lifetime. It leads to the following unsafe behaviors:

- Use after free and dangling pointers (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Double frees (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Buffer overflows and out of bound access (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Iterator invalidation (Stroustrup 2013).

- Data races (Stroustrup 2013).
- Uninitialized memory reads (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Reliance on undefined behaviors (Stroustrup 2013).

Industry data confirms that memory-unsafe languages (primarily C and C++) account for the majority of security vulnerabilities. Google reports that 70% of Chrome vulnerabilities stem from memory safety bugs (*Memory Safety in Android and Chrome* 2022), and Microsoft reports similar figures across Windows components (*70% of Security Bugs Are Memory Safety Issues* 2020). Although modern C++ attempts to mitigate issues through smart pointers and safer API design, legacy code and raw pointer usage remain widespread (CppCon 2020).

C++ concurrency and data races

C++ has a standardized model of memory since C++ 11. Through Standard C++ library (STD), C++ provides access to threads (for multi processing and for parallel execution) and atomics that are a powerful mechanism for managing concurrent access to shared data in multithreaded applications. Also provides mutexes, locks and condition variables for safety memory access for multiple processes concurrently.

With all those features C++ provides a possibility of an high performance for parallel programs, but the correctness of C++ remains difficult. That is due C++ not being capable of statically prevent data races, C++ allows aliasing and mutation via multiple paths, exposes low level memory order operations and relies on the developer to ensure invariants (Stroustrup 2020).

At following example is possible to see two threads modifying the same resource, the global variable ***counter***. The modification is concurrently leading to a undefined behavior.

```
1   #include <iostream>
2   #include <thread>
3   #include <vector>
4
5   int counter = 0; // Shared resource
6
7   void increment() {
8       for (int i = 0; i < 1000000; ++i) {
9           // Data race occurs here: read-modify-write is NOT atomic
10          counter++;
11      }
12  }
13
14  int main() {
15      std::thread t1(increment);
16      std::thread t2(increment);
17
18      t1.join();
19      t2.join();
```

```
20
21          std::cout << "Final counter: " << counter << std::endl;
22          // Expected: 2000000
23          // Actual: Likely < 2000000 (e.g., 146377) due to race conditions
        [web:51]
24          return 0;
25  }
```

The incremental of ***counter***, **counter++**, is not atomic. Usualy an incremental operation includes three steps: the load of the variable into a register, the value increment at the register and the value store back at the memory.

When this operation is not protected by locking (e.g. using threads) or atomic types (**std::atomic<int>**), both threads can load the same value from memory, increment it and store it, simultaneously. Since the initial value is the same, instead of expected increment by 2 it only will increment by 1.

2.2.2 Boost C++ Libraries

The Boost C++ Libraries are a comprehensive collection of peer-reviewed, portable, and open-source libraries that function as a testbed for the C++ Standard Library. Boost has heavily influenced modern C++ standards (C++11 through C++23), serving as the incubator for foundational components such as smart pointers, threading models, and regular expressions. Boost has a rigorous peer review process and commitment to portability made it a benchmark for high quality C++ library development. The capacity of early provision of production ready implementations of libraries, enabled developers to adopt it. (*Boost C++ Libraries Documentation* 2025)

2.2.3 The Boost Paradox: Barrier and Bridge

The Boost libraries have, historically, served as the proving ground for the C++ Standard Library. Boost provide high-quality, peer-reviewed, and largely header-only components that target portability and performance across platforms. (*Boost C++ Libraries Documentation* 2025) This ubiquity creates a unique dichotomy during migration, because the same abstractions that enabled modern C++ design patterns also amplify the complexity of introducing a Rust-based ownership model into existing codebases.

- **Technical mismatch**: Boost utilizes advanced C++ features such as template metaprogramming, complex inheritance hierarchies, SFINAE (Substitution Failure Is Not An Error) that is a principal in C++ where an invalid substitution of template parameters is not an error in itself, and expression templates across libraries as `Boost.SmartPtr`, `Boost.Spirit`, and `Boost.Asio` (*Boost C++ Libraries Documentation* 2025).

These constructs don't have direct equivalents in Rust's generic system and trait bounds, particularly when combined with partial specialization and ADL-dependent overload sets. As a consequence, standard automated binding tools (such as `bindgen` (The rust-bindgen Project Developers 2026)) are ill-suited to interface with Boost's header-only, template-heavy design: the absence of concrete, non-templated symbols at the ABI level often forces developers to introduce intermediate C-compatible facades.

These facades are typically handwritten wrappers that manually erase templates into plain C types or opaque handles, reintroducing the same classes of human error (lifetime mismatches, missing error propagation, accidental copying) that Rust aims to eliminate in the first place.

This code uses a template, `async_wait` from the library boost::asio. It is not a compiled symbol in the library, is generated only when used. Rust is not able to link to it directly.

```
1       // async_engine.hpp
2       #include <boost/asio.hpp>
3       #include <iostream>
4
5       class AsyncEngine {
6       public:
7           AsyncEngine() : timer_(io_, boost::asio::chrono::
    milliseconds(500)) {}
8
9           // PROBLEM: This method uses a C++ Template for the
    handler (Lambda).
10          // Bindgen ignores this because it requires template
    instantiation.
11          // It relies on SFINAE to check if the handler has the
    right signature.
12          template <typename CompletionToken>
13          void start_task(CompletionToken&& token) {
14              timer_.async_wait(token);
15              io_.run();
16          }
17
18      private:
19          boost::asio::io_context io_;
20          boost::asio::steady_timer timer_;
21      };
22
```

- **Semantic opportunity**: Many Boost libraries were explicitly designed to encode disciplined resource management and concurrency patterns, that anticipate concerns later

formalized by Rust. Boost.SmartPtr library encapsulates ownership and shared lifetime through types, such as `boost::shared_ptr` and `boost::scoped_ptr`.

The library Boost.Thread provides higher-level threading primitives and synchronization constructs beyond the C++03 baseline (*Boost C++ Libraries Documentation* 2025).

Higher-level facilities, such as `Boost.Program_options` and `Boost.Asio`, encourage clear separation of configuration, I/O and business logic. It aligns well with Rust's preference for explicit data flow and minimal global state.

The migration opportunity lies in systematically leveraging these "vocabulary types" as a pivot layer: raw pointers and ad hoc ownership conventions at subsystem boundaries are first refactored into Boost abstractions, which can then be mechanically mapped to Rust equivalents (for example, `boost::shared_ptr<T>` → `std::sync::Arc<T>`, `boost::optional<T>` → `Option<T>`, or `boost::variant` → a Rust `enum`).

Treating Boost as a semantic bridge rather than a legacy liability allows the migration process to progressively tighten ownership and error-handling contracts on the C++ side before introducing Rust, thereby reducing the impedance mismatch at the FFI boundary.

A simulation of a shared configuration system based, it is usualy used in ROS/robotics systems.

```
1        #include <vector>
2        #include <string>
3        #include <iostream>
4        #include <boost/variant.hpp>
5        #include <boost/shared_ptr.hpp>
6        #include <boost/make_shared.hpp>
7
8        // 1. Define the sum type: It's EITHER an int OR a string
9        typedef boost::variant<int, std::string> ConfigValue;
10
11       // 2. Define the container: A shared list of these values
12       typedef std::vector<ConfigValue> ConfigList;
13       typedef boost::shared_ptr<ConfigList> SharedConfig;
14
15       // Visitor to print values (Required to safely unpack the
    variant)
16       struct Printer : public boost::static_visitor<void> {
17           void operator()(int i) const { std::cout << "Speed: " <<
     i << std::endl; }
18           void operator()(const std::string& s) const { std::cout
    << "Mode: " << s << std::endl; }
19       };
20
21       int main() {
22           // 3. Create shared data
```

```
23              SharedConfig config = boost::make_shared<ConfigList>();
24              config->push_back(100);             // Add int
25              config->push_back("Autonomous");    // Add string
26
27              // 4. Share it (cheap copy, points to same data)
28              SharedConfig worker_ref = config;
29
30              // 5. Iterate and Visit (Type-safe access)
31              for (const auto& item : *worker_ref) {
32                  boost::apply_visitor(Printer(), item);
33              }
34
35              return 0;
36          }
37
38          // Rust equivalent using built-in equivalents (enum and
    match) instead of library templates
39          use std::sync::Arc;
40
41          // 1. Define the sum type (Enum)
42          enum ConfigValue {
43              Speed(i32),
44              Mode(String),
45          }
46
47          fn main() {
48              // 2. Create shared data (Vec wrapped in Arc)
49              // Note: Rust requires explicit variants (ConfigValue::
    Speed) unlike Boost
50              let config = Arc::new(vec![
51                  ConfigValue::Speed(100),
52                  ConfigValue::Mode(String::from("Autonomous")),
53              ]);
54
55              // 3. Share it (cheap clone, points to same data)
56              let worker_ref = config.clone();
57
58              // 4. Iterate and Match (Type-safe access)
59              for item in worker_ref.iter() {
60                  match item {
61                      ConfigValue::Speed(i) => println!("Speed: {}", i
    ),
62                      ConfigValue::Mode(s) => println!("Mode: {}", s),
63                  }
64              }
65          }
66
```

[language=C++]

2.3 Interoperability between Rust and C++

The transition from C++ to Rust represents a necessary evolution for high-assurance software, driven by the critical need to eliminate memory safety vulnerabilities characteristic of C++'s manual memory management. However, for systems deeply rooted in the C++ ecosystem, particularly those dependent on the Boost C++ Libraries, migration is not merely a syntactic translation but a complex architectural reconciliation. The core challenge lies in bridging two distinct paradigms without sacrificing the stability of massive legacy investments.

2.3.1 Undefined Behavior and Vulnerability Mitigation

A primary driver for this migration is the prevalence of undefined behavior in legacy C/C++ codebases, which often manifests as security vulnerabilities. A best-case historical study of open-source vulnerabilities indicates that approximately 58.2% of historical vulnerabilities in prominent C projects would have been virtually impossible in safe Rust. (Meneely et al. 2025)

Code vulnerabilities are formally listed at Common Weakness Enumeration (CWE). CWE was community developed and list common software and hardware weakness types.

Some critical vulnerabilities detected were:

- **Use-after-free (CWE-416)**: While Boost smart pointers manage object lifetimes, they cannot strictly enforce compile-time borrow checking, leaving potential for dangling references if raw pointers are extracted. Rust's borrow checker eliminates this entirely in safe code.
- **Buffer overflows (CWE-120/125)**: C++ containers (including `std::vector` and `boost::container`) rely on developer discipline to avoid out-of-bounds access. In contrast, Rust enforces bounds checking by default, preventing memory corruption.
- **Data races (CWE-362)**: Rust's `Send` and `Sync` traits ensure thread safety at the type level, preventing data races that are common in complex C++ multithreaded applications, even when using `boost::thread`.

However, automated translation tools often fail to fully capitalize on these safety guarantees. Transpilers like C2Rust may preserve the unsafe semantics of the original C code, necessitating further static analysis or refactoring to achieve true safety. (Hong 2023; Ling et al. 2022)

2.3.2 The Safety Air Gap in Hybrid Architectures

Hybrid architectures are structural design patterns that allow two languages to coexist in a single system, as systems with C++ and Rust. They share the same low-level control but

have different compilation models and safety guarantees. Due to it the architecture should focus on isolating the boundary to minimize friction.

Since big-bang rewrites, a software strategy where a system is rebuilt from scratch (rewriting a C++ system in Rust), are resource-prohibitive, organizations are forced into incremental migration.

This creates a prolonged hybrid state where Rust modules must coexist with legacy C++:

- **FFI vulnerabilities**: Rust's safety guarantees (ownership, borrow checking) strictly end at the Foreign Function Interface (FFI) boundary. Crossing this boundary destroys safety unless rigorously wrapped, creating a safety air gap. Dynamic analysis tools like Rustcheck have been proposed to detect memory safety issues specifically arising from unsafe Rust usage at these boundaries. (Xia, Wu, and Hua 2023)
- **Integration risks**: Passing complex Boost data structures across the FFI boundary introduces risks of unsafe pointer usage, exception mismatches, and memory leaks.
- **Concurrency conflicts**: Bridging Boost.Asio or Boost.Thread with Rust's concurrency model can lead to undefined behavior, specifically when distinct async runtimes or threading models attempt to interoperate without a unified strategy.

2.3.3 Systematic Porting vs. the Rewrite Trap

A lack of structured methodology often leads to the rewrite trap stalled development due to the sheer complexity of porting feature-rich systems.

- **Performance overhead**: FFI boundaries introduce latency (10–20% overhead in some cases) due to data marshalling. In latency-sensitive domains (HPC, HFT), an ad hoc porting strategy that places FFI boundaries in hot loops is unacceptable.
- **Missing methodology**: There is currently no standardized process for decomposing a Boost-heavy application into portable units. Developers need a systematic way to identify architectural seams where the cost of the FFI boundary is minimized and the safety value of Rust is maximized.

The central problem is not just the translation of code, but the architectural interoperability of two diverging memory models. The challenge is to define a strategy that stops treating Boost as legacy debt to be discarded and starts utilizing it as a stabilizing intermediate layer.

2.3.4 Foreign Function Interface (FFI)

A Foreign Function Interface (FFI) is a mechanism that enables a program written in one programming language (the host language) to call routines or make use of services written in another language (the guest language). FFIs are critical for system interoperability, allowing high-level languages to leverage legacy codebases, operating system APIs, and

high-performance libraries often implemented in C or C++. (*C++ ABI Compatibility* 2025) In compiled languages like Rust, FFI is often handled through static declarations. The `extern` block defines function signatures that adhere to the C application binary interface (ABI) and Rust uses attributes such as `#[link(name = "...")]` together with `#[repr(C)]` on data structures to ensure layout compatibility and correct symbol resolution. (*Rust Documentation* 2025)

2.3.5 Interoperability tools

There are two main tools for interoperability between C/C++ and Rust: **cxx** and **autocxx**.

Cxx provides a safe mechanism for calling C++ code from Rust and Rust code from C++, not subject to undefined behaviors and memory safety risks often introduced when using **bindgen** or **cbindgen** to generate C-style bindings. Unfortunately, beside this, C++ code remains 100% unsafe.

AutoCxx is a tool for calling C++ from Rust in a heavily automated form. It includes all fluent safety from Cxx generating interfaces automatically from C++ headers using a variant of **bindgen**.

2.3.6 Boost migration for Rust

Currently there is no official migration of Boost from C++ to Rust. Both ecosystems, C++ and Rust, are designed within different paradigms. Boost is described as a peer reviewed open source repository of C++ libraries. It is a massive, centralized collection that is used as testing ground for features eventually destined for STD.

Rust follows a different model. Instead of having a massive "Boost" crate, the community builds specialized, high-quality crates that are distributed through crates.io.

Boost, as previous reference, has an heavy use of Template Metaprogramming (TMP). It is not straightforward translation, the porting to Rust. That is due the fact of C++ and Rust handle generics differently. C++ templates are checked at compile time, if the code works. Rust generics are definition checked. Boost also relies on complex class hierarchies and inheritance. On other side, Rust uses composition, meaning a total architectural rewrite for Rust implementation of Boost. Boost also uses frequently smart pointers to manage complex object graphs and Rust uses ownership and borrowing that is a completely different approach. Those Rust rules are used to avoid the overhead of reference counting.

There is an entry at Crates.io for porting Boost libraries to Rust but there is no updates since 2021. At this moment has no source code (Zakarum 2023). On other side, there are some Rust native equivalents to Boost C++ at Crates.io that are safer and faster. At the following table is possible to check some equivalents (*crates.io: The Rust Community's Crate Registry* 2025).

Boost Library	Rust Equivalent	Notes
Boost.Asio	`tokio`, `async-std`	Async IO, networking, event loop
Boost.Beast	`hyper`, `reqwest`	HTTP clientserver, WebSockets
Boost.Filesystem	`std::fs`, `std::path`	Native filesystem support in Rust stdlib
Boost.ProgramOptions	`clap`, `argh`, `pico-args`	Command-line parsing
Boost.Serialization	`serde`	General-purpose serialization framework
Boost.Regex	`regex`	Fast regular expression engine
Boost.Optional	`Option<T>`	Built-in language feature
Boost.Variant	`enum`, `enum_dispatch`	Tagged unions sum types
Boost.Any	`dyn Any`	Type-erased storage
Boost.Graph	`petgraph`	Graph algorithms and data structures
Boost.Geometry	`geo`, `geo-types`	Computational geometry
Boost.SmartPtr	`Box`, `Rc`, `Arc`	Memory management primitives
Boost.Thread	`std::thread`, `crossbeam`	Threading and concurrency
Boost.Lockfree	`crossbeam`, `loom`	Lock-free data structures
Boost.Coroutine	`async/await` (built-in), `futures`	Cooperative concurrency

TABLE 2.1: Rust ecosystem crates that replace common Boost C++ Libraries.

2.4 Case studies

After checking the IEEE paper library were not found specific case studies that directly migrate or benchmark Boost C++ libraries. The papers mainly focus on migrating C codebases or in specific FFI algorithms.

2.4.1 Concurrency Safety: Migrating Locking primitives

Legacy C software often uses `pthread_mutex_lock`/`pthread_mutex_unlock` to control concurrency. If C2Rust is used to migrate the C code directly to Rust, the resulting code still contains unsafe C stlyle mutex operations, preventing Rust compiler from enforcing thread-safety guarantees (Immunant, Inc. 2025).

Was proposed, by Jaemin Hong, a solution to automatically migrate C programs to Rust with an high focus on concurrency. In order to solve C2Rust problems, was proposed a solution named Concrat. The solution Concrat consists on a aggregation of a C2Rust solution, a static analyzer and a Rust code transformer. The static analyzer performs the dataflow analysis of the code generated by the C2Rust to produce a summary about data and flow lock. The transformer replaces the lock API according to the previous summary.

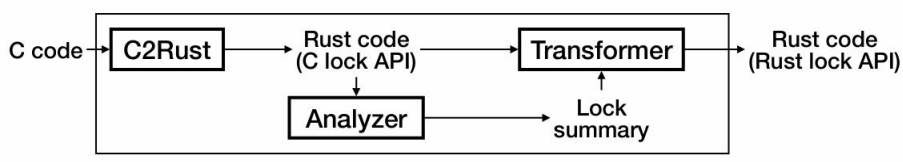

FIGURE 2.1: Concrat workflow (Hong 2023)

The tool achieved valid compilable Rust code in 29 of 46 cases collected from the github, proving that ownership-based concurrency can be automated even for legacy codebases (Hong 2023). From the 46 cases, 5 of them were also compilable but with a need of a manual fixes. The remain 12 cases were considered failures classified into three categories: functions conditionally acquiring locks, function pointers, and pointers to locks (Hong 2023).

Concrat focuses on C rather than C++, although its findings are applicable at C++ and also applicable to Boost C++ migration. Libraries as Boost.Thread and Boost.Asio have a strong dependency on manual synchronization, raw pointers and complex ownership patterns. The study has demonstrated that a simple translation of that code to Rust, relying on C2Rust or C++ bindings, would preserve the safety risks unless concurrency semantics are explicitly reconstructed. Concrat's static analysis offers a practical blueprint for the reconstruction providing a lock analysis.

This case study illustrates how automated, semantics-aware transformation can bridge the gap between C++'s flexible but unsafe concurrency model and Rust's strict, verifiable approach, enabling incremental and safer integration of Rust into existing C++ ecosystems.

2.4.2 "Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C

In this paper the authors provide a historical security analysis of C language projects, based on the most common vulnerabilities of C and provide answers Rust based for those vulnerabilities. Those vulnerabilities were divide into two groups: the vulnerabilities Rust could prevent and the vulnerabilities Rust cannot prevent. Within this paper 68 prominent open source projects, as linux kernel and Curl, were analyzed in order to map the vulnerabilities.

The study identified a total of 4,507 vulnerabilities across these projects, 2,257 of which had specific Common Weakness Enumeration CWE designations. These were mapped to Rust's safety guarantees to determine preventative potential. The analysis reveals that a significant majority of historical security issues in C are memory safety errors that Rust effectively eliminates at compile time.

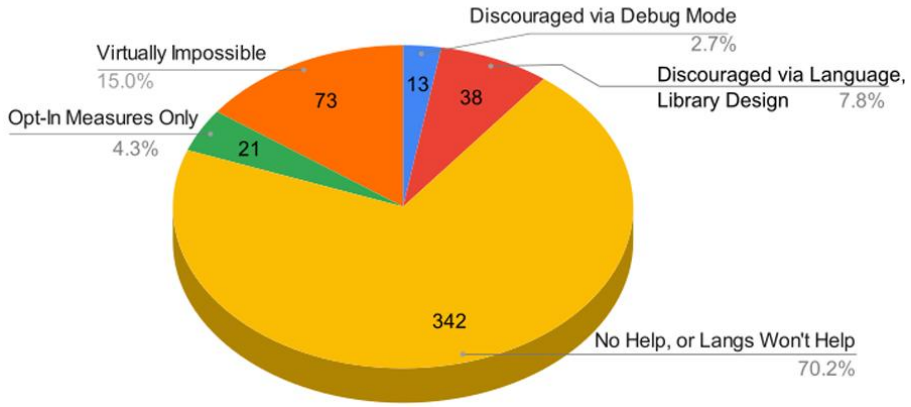

FIGURE 2.2: Rust Preventions based on CWE (Meneely et al. 2025)

As illustrated in Figure 2.2, the vulnerabilities are distributed across several categories of potential prevention through Rust. Approximately 15.0% fall into the "Virtually Impossible" category, indicating vulnerabilities that can be prevented by Rust's type system and ownership model. In contrast, approximately 70.2% fall into the "No Help, or Langs Won't Help" category, indicating vulnerabilities that cannot be prevented solely through the choice of programming language. The remaining vulnerabilities fall into intermediate categories where prevention depends on language features, libraries, compiler configurations, or development practices (Meneely et al. 2025).

Table 1 details the top 20 most common vulnerability types found in the subject projects. The data confirms that the two most frequent issues, NULL Pointer Dereference (CWE-476) and Use After Free (CWE-416), accounting for nearly 17% of all analyzed CVEs, are completely preventable in safe Rust (Meneely et al. 2025).

CWE ID	Vulnerability Name	# CVEs	# Proj	Rust Prevention
CWE-476	NULL Pointer Dereference	411	17	Virtually Impossible
CWE-416	Use After Free	343	17	Virtually Impossible
CWE-125	Out-of-bounds Read	145	14	Virtually Impossible
CWE-20	Improper Input Validation	105	17	No Help
CWE-190	Integer Overflow or Wraparound	100	12	Discouraged (Debug)
CWE-401	Memory Leak	96	6	Virtually Impossible
CWE-787	Out-of-bounds Write	92	15	Virtually Impossible
CWE-667	Improper Locking	86	4	Opt-In Measures
CWE-369	Divide By Zero	78	4	Virtually Impossible
CWE-617	Reachable Assertion	63	6	Discouraged (Debug)
CWE-362	Race Condition	62	11	Discouraged (Design)
CWE-119	Improper Buffer Restriction	62	14	Virtually Impossible
CWE-400	Uncontrolled Resource Consumption	59	15	No Help
CWE-122	Heap-based Buffer Overflow	53	12	Virtually Impossible
CWE-200	Exposure of Sensitive Information	45	11	No Help
CWE-120	Classic Buffer Overflow	45	9	Virtually Impossible
CWE-908	Use of Uninitialized Resource	35	5	Virtually Impossible
CWE-415	Double Free	34	8	Virtually Impossible
CWE-770	Resource Allocation without Limits	29	7	No Help
CWE-131	Incorrect Buffer Size Calculation	26	7	Virtually Impossible

TABLE 2.2: Top 20 Most Common Vulnerabilities and Rust Prevention Status (Meneely et al. 2025)

This data suggests that while Rust is not a "silver bullet" for all security issues (such as input validation or logic bugs), its adoption could have theoretically prevented the majority of the most severe historical vulnerabilities found in these critical infrastructure projects (Meneely et al. 2025).

Chapter 3

Methodology and Solution Proposal

The migration of critical software infrastructure from C++ to Rust is not a syntactic translation exercise but a fundamental architectural transformation. It requires rethinking ownership, concurrency, and interface boundaries in order to preserve functionality while improving safety guarantees.

As discussed in Chapter 2, the so-called “Boost Paradox” introduces a unique challenge: the very features that make Boost powerful—such as templates and metaprogramming—also hinder the applicability of automated interoperability tools.

This chapter defines a structured methodology to decompose Boost-based systems and proposes an architectural design that incrementally bridges the safety gap between C++ and Rust.

3.1 Component Selection Criteria

Historically, a “big bang” rewrite of large C++ codebases into Rust has proven prone to failure due to the difficulty of verifying feature parity, the disruption of ongoing development, and the elevated certification risk in safety-critical domains. Consequently, this research adopts a selective and incremental migration strategy.

Boost libraries are not considered uniformly; instead, they are evaluated and selected based on their relevance, isolation capability, and suitability for automotive software architectures.

3.1.1 Automotive-Oriented Selection Rationale

The automotive sector has increasingly adopted open-source and modular software platforms to address growing system complexity and multi-vendor integration. In this context, several automotive manufacturers and suppliers participate in Eclipse Software Defined Vehicle initiatives, including Eclipse Safety-Critical Open Research Environment (SCORE).

Eclipse SCORE targets deterministic, portable, and interoperable software architectures with long-term support guarantees—key concerns for ISO-26262 compliant systems (Foundation

2025). These initiatives reflect the realities of modern automotive development: large, heterogeneous C++ codebases, long product lifecycles, and strict constraints on timing, memory usage, and failure modes. Such characteristics render wholesale language replacement infeasible and reinforce the need for incremental, well-contained migration strategies.

The Boost libraries selected for migration are therefore evaluated according to the following automotive-oriented criteria:

- **Relevance to in-vehicle software**: Preference is given to libraries commonly used in middleware, communication, configuration management, and concurrency control, which are prevalent in automotive Electronic Control Units (ECUs) and service-oriented vehicle architectures.
- **Deterministic behavior**: Libraries whose runtime behavior is predictable and suitable for real-time or near-real-time constraints are prioritized, aligning with automotive requirements for bounded latency and reproducible execution.
- **Safety and robustness**: Components that encode disciplined resource management or concurrency patterns (e.g., Boost.SmartPtr, Boost.Thread, Boost.Asio) are preferred, as they conceptually align with Rust's ownership and concurrency models.
- **Isolation capability**: Libraries that can be encapsulated behind stable, well-defined interfaces are favored, allowing them to serve as clear boundaries for Foreign Function Interface (FFI) integration.
- **Industrial adoption and tooling maturity**: Libraries actively used or referenced in automotive-oriented open-source projects, such as those under the Eclipse SCORE ecosystem, are considered more suitable due to their demonstrated industrial relevance.

3.1.2 Deprioritized Components

Not all Boost libraries are suitable for migration or interoperability with Rust. Libraries that rely heavily on advanced template metaprogramming, domain-specific embedded languages, or extensive compile-time code generation are deprioritized due to their limited analyzability and poor suitability for FFI-based integration.

Similarly, libraries not directly used or referenced in automotive-oriented open-source projects such as Eclipse SCORE are not considered immediate migration candidates within the scope of this work.

3.1.3 Safety-Critical Constraints and ISO 26262 Alignment

ISO-26262 requires that software architectures for automotive safety systems support freedom from interference, predictable execution, and systematic fault avoidance. Programming

languages and libraries used in such systems must minimize undefined behavior, provide explicit ownership semantics, and enable static verification wherever possible (ISO 2018).

C++'s permissive memory model and reliance on developer discipline pose challenges when targeting higher Automotive Safety Integrity Levels (ASIL) levels. In practice, Boost libraries are frequently employed in automotive systems to impose structure and consistency on C++ development, particularly in resource management, concurrency, and asynchronous communication (Synopsys 2025).

The proposed methodology leverages this disciplined use of Boost as a preparatory step toward Rust integration. By constraining C++ systems to well-defined Boost abstractions, reliance on raw pointers, implicit ownership conventions, and ad hoc concurrency mechanisms is reduced. This aligns with ISO-26262 recommendations for limiting systematic faults through controlled language subsets, coding guidelines, and architectural enforcement (ISO 2018).

Rust components introduced under this methodology are designed to be ownership-complete and memory-safe by construction. Unsafe code is strictly isolated at Foreign Function Interface (FFI) boundaries, supporting safety argumentation, auditability, and certification activities.

3.1.4 Boost Libraries Relevant to Eclipse SCORE

In Eclipse SCORE-style automotive architectures, Boost libraries are commonly used to support middleware services, configuration management, concurrency, and asynchronous I/O. Their selection is driven by the need to enforce structure and predictability in large-scale, safety-relevant C++ systems.

Table 3.1 summarizes Boost libraries commonly applicable to automotive platforms such as Eclipse SCORE and their corresponding Rust-native equivalents.

This mapping highlights that several Boost libraries already encode concepts such as explicit ownership, variant-based state, and structured concurrency— concepts that are native to Rust. As a result, the primary challenge of migration is architectural rather than conceptual, reinforcing the feasibility of incremental Rust adoption in safety-critical automotive ecosystems.

3.1.5 Comparative Analysis as a Migration Enabler

A central step in the proposed methodology is the comparison between Boost libraries used in automotive systems and their Rust-native equivalents. This comparison is not a simple replacement exercise, but an architectural analysis that exposes semantic gaps, safety assumptions, and ecosystem limitations.

By explicitly mapping Boost abstractions to Rust constructs, developers can:

Boost Library	Rust Equivalent	Automotive Relevance
Boost.Asio	`tokio`, `async-std`	Asynchronous I/O and event handling for in-vehicle services and middleware; requires careful runtime integration in safety-critical contexts
Boost.Thread	`std::thread`, `crossbeam`	Thread management and synchronization for concurrent ECU software components
Boost.SmartPtr	`Box`, `Rc`, `Arc`	Explicit ownership and lifetime management; directly maps to Rust's ownership model
Boost.Optional	`Option<T>`	Explicit representation of optional values, reducing NULL pointer usage
Boost.Variant	`enum`	Type-safe sum types used in configuration, messaging, and state machines
Boost.Program_options	`clap`, `argh`	Configuration parsing for tooling and non-safety-critical control components
Boost.Filesystem	`std::fs`, `std::path`	File and path handling in diagnostic, logging, and update subsystems
Boost.Chrono	`std::time`	Time measurement and timeout handling with deterministic semantics

TABLE 3.1: Boost libraries relevant to Eclipse SCORE–style automotive systems and Rust equivalents

- Identify which safety properties are already enforced at the type level
- Detect mismatches in ownership, error handling, or concurrency semantics
- Quantify the residual risk that remains after migration

This analysis also reveals areas where no suitable Rust equivalent exists for automotive-grade use. In such cases, the absence of a mature Rust library becomes an explicit design signal rather than an implementation obstacle.

3.1.6 From Library Gaps to New Rust Automotive Crates

The comparison between Boost and Rust ecosystems frequently uncovers functionality that is either missing or insufficiently mature in Rust for safety-critical automotive contexts.

Rather than forcing unsafe bindings or retaining C++ dependencies indefinitely, the proposed methodology treats these gaps as opportunities for new Rust library development. Such libraries are designed from the outset with:

- Explicit ownership and lifetime semantics
- Deterministic execution models suitable for real-time analysis

- Minimal and auditable unsafe code
- Alignment with AUTOSAR Adaptive concepts

In this sense, Boost serves not only as a migration bridge, but also as a functional benchmark that informs the evolution of Rust's automotive ecosystem. The migration process thus becomes bidirectional: existing C++ designs guide Rust library development, while Rust's guarantees inform safer architectural patterns in C++.

3.1.7 ASIL-Oriented Migration Strategy

ISO-26262 distinguishes software components according to their ASIL, ranging from Quality Management (QM) to ASIL-D. Each level imposes increasing requirements on fault avoidance, fault detection, and freedom from interference. As a result, a uniform migration strategy is neither practical nor desirable across all ASIL levels.

The proposed methodology adopts a differentiated migration approach:

- **QM and ASIL-A components**: Tooling, diagnostics, logging, configuration services, and non-safety-critical middleware. These components are suitable early candidates for Rust adoption, as certification constraints are less restrictive.
- **ASIL-B and ASIL-C components**: Communication services, resource managers, and coordination logic. Rust integration at this level focuses on eliminating memory safety defects while maintaining deterministic execution. Unsafe Rust is strictly isolated and subject to additional verification.
- **ASIL-D components**: Rust is introduced only where its guarantees provide demonstrable safety benefits. Components must be ownership-complete, free from dynamic allocation where required, and analyzable under Worst Case Execution Time (WCET) constraints. FFI boundaries are minimized and treated as safety-relevant interfaces subject to rigorous review.

This stratification aligns with ISO-26262 recommendations for mixed-criticality systems and supports freedom from interference between software elements of different ASIL levels.

3.1.8 Migration Workflow Overview

Figure 3.1 presents the proposed incremental migration workflow for integrating Rust into Boost-based automotive software systems. The workflow is designed to support safety-critical development under ISO-26262 constraints while preserving system continuity and certification traceability.

The process begins with a **comparative analysis** of existing Boost library usage and available Rust-native equivalents. This step establishes a semantic baseline by identifying ownership models, concurrency assumptions, error-handling mechanisms, and determinism properties already enforced in the C++ architecture.

Based on this analysis, an **architectural refactoring phase** follows, during which Boost abstractions are used to constrain C++ components into well-defined, analyzable interfaces. This step reduces semantic ambiguity, limits undefined behavior, and prepares the system for language boundary introduction without altering functional behavior.

Next, **FFI boundary definition** is performed. Stable and minimal interfaces are identified between C++ and Rust components, with explicit ownership transfer rules and data lifetime guarantees. These boundaries are treated as safety-relevant interfaces and are subject to additional review and verification.

The workflow then proceeds with **incremental Rust integration**, where Rust components are introduced selectively according to their assigned ASIL level. Initial integration targets QM and ASIL-A components, followed by progressively higher-criticality elements as confidence, tooling maturity, and safety argumentation evolve.

Throughout the process, **continuous verification and validation** are performed to ensure functional equivalence, timing predictability, and freedom from interference. Feedback from each iteration informs subsequent migration steps, allowing the workflow to adapt to architectural constraints and certification requirements.

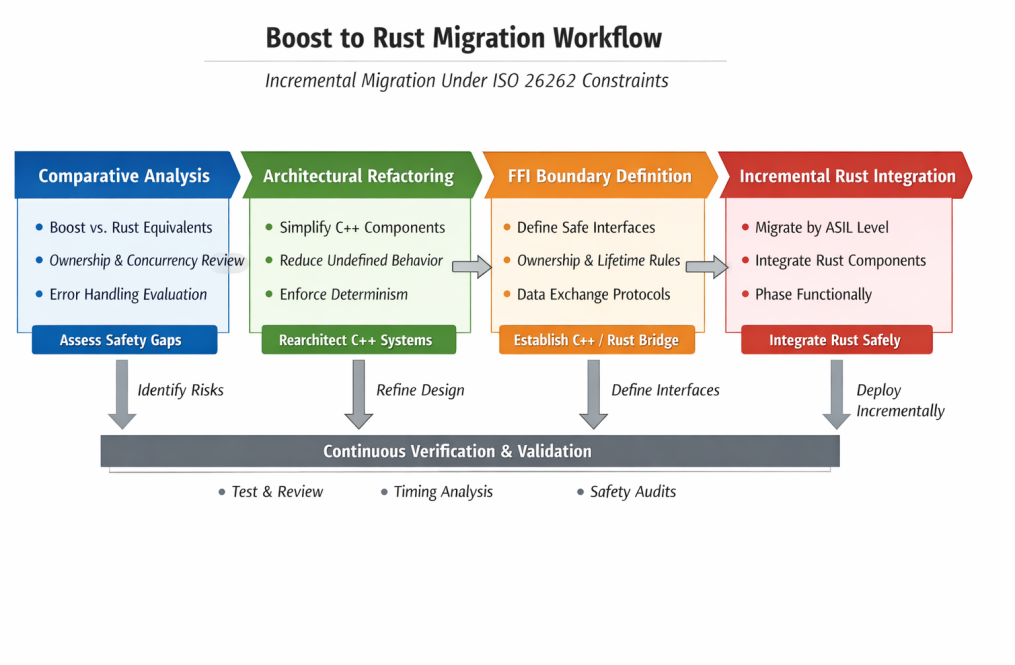

FIGURE 3.1: Incremental migration workflow for Boost-based automotive systems integrating Rust under ISO-26262 constraints

Overall, the workflow emphasizes early reduction of semantic ambiguity on the C++ side and controlled introduction of Rust components, enabling safer and more predictable interoperability while maintaining compliance with automotive safety standards.

Chapter 4

Development Plan

This chapter presents the development plan for the proposed Boost-to-Rust migration methodology. It details the planned reimplementation of selected components in Rust, the design of stable interfaces, and the integration of bidirectional FFI. The plan is structured to support incremental adoption while maintaining compliance with automotive safety standards such as ISO-26262 and architectural principles from AUTOSAR Adaptive.

The development activities are scheduled between February and July and are organized to progressively reduce technical risk while preserving functional continuity of the existing C++ system.

4.1 Objectives and Scope

The primary objectives of this development phase are:

- To reimplement selected Boost-based components in Rust with equivalent functionality and improved memory-safety guarantees.
- To design explicit, minimal, and auditable interfaces between C++ and Rust components.
- To integrate Rust modules incrementally using bidirectional FFI, preserving deterministic behavior and freedom from interference.
- To validate the resulting hybrid system against safety, correctness, and performance expectations relevant to automotive software.

The scope of this chapter is limited to architectural and integration-level development. Functional feature expansion beyond parity with the existing C++ implementation is explicitly excluded.

4.2 Development Phases Overview

The development plan is divided into six sequential phases:

1. System analysis and Boost component selection
2. Architectural refactoring and interface definition
3. Rust reimplementation of selected components
4. Bidirectional FFI integration
5. Validation and safety analysis
6. Consolidation and documentation

Each phase produces concrete artifacts that serve as inputs for the subsequent phase, enabling traceability and controlled progression consistent with ISO-26262 development practices.

4.3 System Analysis and Component Selection

February

The initial phase focuses on analyzing the existing C++ system and identifying Boost libraries that are relevant to automotive software architectures such as those used in Eclipse SCORE–based environments.

The analysis includes:

- Identification of Boost libraries in use and their functional roles
- Classification of components by criticality and ASIL level
- Mapping of Boost abstractions to potential Rust-native equivalents

This phase establishes the baseline architecture and defines the migration targets for subsequent development.

4.4 Architectural Refactoring and Interface Design

March

In this phase, the selected C++ components are refactored to reduce semantic ambiguity and prepare them for interoperability. Boost abstractions are used to replace raw pointers, ad hoc ownership conventions, and implicit lifetimes.

Key activities include:

- Encapsulation of migrated components behind stable interfaces
- Elimination of undefined behavior patterns
- Definition of ownership and lifetime semantics at component boundaries

This refactoring step does not change observable behavior but improves analyzability and safety argumentation.

4.5 Rust Reimplementation

April

During this phase, selected Boost-based components are reimplemented in Rust. The Rust implementations are designed to be ownership-complete and memory-safe by construction.

Design constraints include:

- Explicit ownership and borrowing semantics
- Deterministic execution suitable for timing analysis
- Minimal use of `unsafe` code, isolated and documented

Functional equivalence with the original C++ components is verified through unit testing and interface-level validation.

4.6 Bidirectional FFI Integration

May

This phase introduces bidirectional FFI integration between C++ and Rust. Stable C-compatible interfaces are defined, and responsibility for memory allocation, ownership transfer, and error handling is made explicit.

Key concerns addressed include:

- Isolation of unsafe code at FFI boundaries
- Prevention of exception and panic propagation across language borders
- Alignment with AUTOSAR Adaptive service-oriented communication models

The FFI boundary is treated as a safety-relevant interface and is subject to additional review.

4.7 Validation and Safety Analysis

June

Validation activities focus on ensuring that the hybrid C++/Rust system meets functional, safety, and performance expectations.

This includes:

- Functional regression testing

- Concurrency and race-condition analysis
- Review of freedom from interference between components of different ASIL levels

The results of this phase support the safety argumentation required for ISO-26262 compliant systems.

4.8 Consolidation and Documentation

July

The final phase consolidates the development results and prepares the system for evaluation and reporting.

Activities include:

- Final architectural documentation
- Evaluation of migration feasibility and limitations
- Documentation of lessons learned and future work

This phase ensures that the development process and outcomes are reproducible and auditable.

4.9 Development Timeline

Table 4.1 presents a Gantt-style overview of the development plan covering the February–July period.

Activity / Month	**Feb**	**Mar**	**Apr**	**May**	**Jun**	**Jul**
System Analysis and Boost Selection	X					
Architectural Refactoring		X				
Rust Reimplementation			X			
Bidirectional FFI Integration				X		
Validation and Safety Analysis					X	
Consolidation and Documentation						X

TABLE 4.1: Gantt-style development plan for the Boost-to-Rust migration

Chapter 5

Migration strategy

This chapter defines a systematic strategy for migrating selected Boost libraries to Rust. The strategy establishes a common methodology that is applicable to each of the selected libraries and provides a structured approach for evaluating the resulting implementations.

The primary objective of the proposed strategy is to reproduce the relevant functionality of the selected Boost libraries in Rust while preserving their expected behavior and enabling their integration with existing C++ codebases. In addition, the strategy considers the requirements and constraints associated with the potential use of the migrated libraries in safety-critical automotive software architectures.

The migration strategy is divided into six main steps, described below:

1. **Understanding the library's functionality and its dependencies.**

   The first step consists of analyzing the selected Boost library's source code, documentation and relevant technical literature, to understand its purpose, functionality, interfaces and dependencies.

   This analysis identifies the relevant classes,funtions, data structures, algorithms, platform-specific behavior and interactions with other libraries. Particular attention is given to aspects that affect the observable behavior of the library and that may influence its migration to Rust.

   A comprehensive understanding of these aspects is essential to ensure that the Rust implementation reproduces the relevant semantics of the original library and can be subsequently be integrated into existing C++ codebases.

2. **Define Rust equivalents for the library's classes, functions, and data structures.**

   Based on the analysus of the library's original implementation, the equivalent Rust abstractions are defined for the library's classes, functions, data structures and interfaces.

   The Rust design does not necessarily replicate the structure of the original C++ implementation. Where appropriate, the design may differ in order to take advantage of

Rust's ownership model, type system, error handling mechanisms and idiomatic programmig practices, while preserving the required behavior of the original library.

3. **Implement the library's functionality in Rust, ensuring that it adheres to Rust's memory safety guarantees and best practices.**

   The library's functionality is reimplemented in Rust. During this stage, particular attention is given to the Rust's ownership and borrowing model, memory safety guarantees, error handling and idiomatic design principles.

   The implemenentation also considers the requirements and constraints of intended target environment. Where the use of `unsafe` Rust is necessary, such usage should be minimized and restricted to well-defined boundaries, particularly when interacting with external code or platform-specific functionality.

4. **Create a C++ wrapper for the Rust implementation, allowing it to be used in existing C++ codebases.**

   To enable integration with existing C++ codebases, an interoperability layer (C++ wrapper) is developed around the Rust implementation.

   The interoperability layer provides an interface suitable for use by the existing C++ applications while isolating the underlying Rust implementation from the consuming code. Particular attention is given to the representation of data, ownership and object lifetime, error handling, memory management and the interaction between the C++ and Rust components.

5. **Test the Rust implementation and the C++ wrapper to ensure that they function correctly and meet performance requirements.**

   The Rust implementation and its C++ wrapper are tested to verify functional correctness, behavioral equivalence with the original Boost library where applicable, interoperability and performance.

   The original Boost implementation serves as a reference when behavioral compatibility is required. Equivalent inputs and relevant execution scenarios are used to identify discrepancies between the original and migrated implementations.

   Performance evaluation is also performed to determine whether the Rust implementation and the interoperability layer introduce unacceptable overhead compared with the original implementation.

6. **Document the migration process, including any challenges encountered and solutions implemented, to provide guidance for future migrations of other Boost libraries to Rust.**

   The final step consists of documenting the migration process, including design decisions, encoutered challenges, solutions, testing results and lessons learned.

This documentation provides a reference for future migrations to other Boost libraries and contributes to the definition of a repeatable migration methodology.

Among these steps, two are particularly critical for the success of the migration strategy. The first is the analysis of the library's functionality and dependencies, since an incomplete understanding of the original implementation can result in incorrect assumptions about its behavior or dependencies. The second is the **implementation of the functionality in Rust**, as this stage requires translating the original C++ abstractions into Rust while preserving the required semantics and taking advantage of Rust's memory safety guarantees and type system. These two stages therefore have a significant influence on the correctness and overall sucess of the migration.

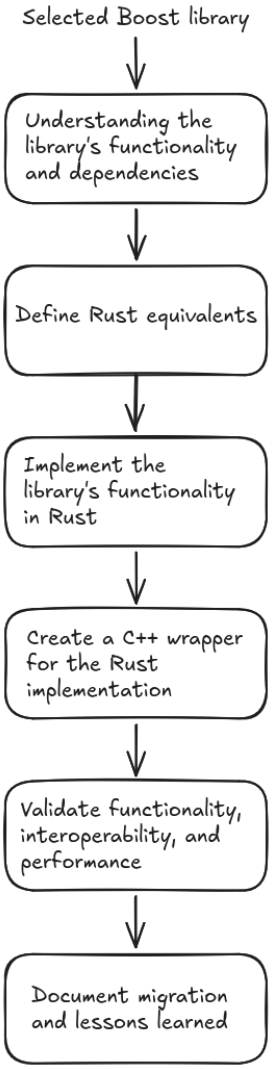

FIGURE 5.1: Migration strategy

The six steps described above can be grouped into four main phases, as follows:

1. **Analysis**
   (a) Understanding the library's functionality and dependencies.
   (b) Defining the migration scope and requirements.
2. **Design and Implementation**
   (a) Defining the Rust equivalents of the relevant C++ abstractions.
   (b) Implementing the required functionality in Rust.
3. **Interoperability**
   (a) Developing the interoperability layer between Rust and C++.
4. **Validation and Evaluation**
   (a) Testing functional correctness, behavioral equivalence, interoperability, and performance.
   (b) Documenting the results, limitations, and lessons learned.

The phases are illustrated in Figure 5.2.

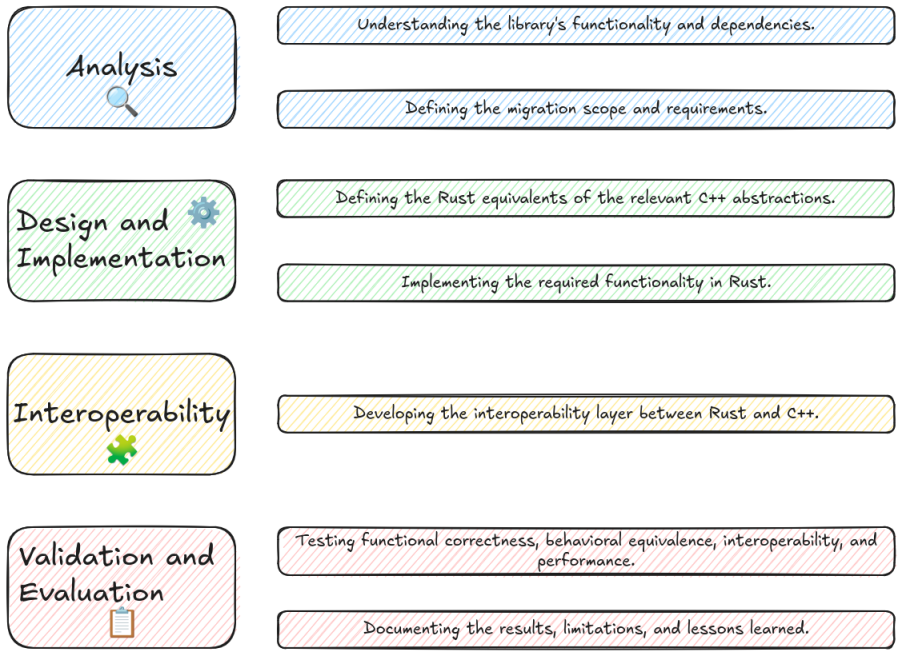

FIGURE 5.2: Migration phases

The migration strategy aims to satisfy the following objectives:

- Preserve the relevant behavior of the original Boost library;
- Provide equivalent functionality through a Rust implementation;
- Maintain interoperability with existing C++ applications;
- Leverage Rust's memory and type safety guarantees;
- Maintain acceptable performance compared with the original implementation;
- Minimize and isolate the use of `unsafe` Rust, particularly at interoperability boundaries;
- Consider the requirements and constraints of the intended automotive environment.

The success of a migration is therefore evaluated according to the extent to which these objectives are satisfied. The specific application of the proposed strategy, including the analysis, implementation, interoperability and evaluation of each selected Boost library, is presented in the following chapter.

Chapter 6

Selected libraries

This chapter presents the selected Boost libraries for reimplementation in Rust. The selection is based on several criteria, including their relevance to automotive software architectures, implementation characteristics, complexity and the potential benefits that a Rust implementation may provide in terms of memory safety, reliability and performance.

The selected libraries are: `Boost.Align` and `Boost.Endian`. Both libraries provide relatively low-level functionality that is relevant to systems programming and embedded software, while also exposing functionality that interacts closely with memory representation and processor architecture. These characteristics make them suitable candidates for evaluating the feasibility of reproducing existing C++ functionality using idiomatic Rust.

For each selected library, this chapter provides an overview of its purpose and functionality, describes its main components and discusses its relevance to automotive software development. The chapter also identifies the main challenges associated with reimplementing the library in Rust, including differences between the C++ and Rust memory models, language abstractions, portability requirements and interoperability with existing C++ codebases.

6.1 Boost.Align

6.1.1 Overview

The `Boost.Align` library provides functions, classes, templates, traits and macros, for controlling, inspecting and diagnosing memory alignment. Its primary purpose is to provide portable alignment-related functionality across different compilers, standard library implementations and language standards.

Memory alignment refers to the requirement that an object is stored at a memory address satisfying a particular alignment constraint. For example, a type with an alignment requirement of 8 bytes must generally be placed at an address divisible by 8. Alignment requirements are determined by the target architecture and by the type being stored. Correct alignment is

important because processors may impose restrictions on the addresses from which particular types can be accessed. Even on architectures that support unaligned accesses, aligned accesses can provide more predictable and efficient memory operations.

Alignment is particularly relevant to low-level and performance-sensitive software. Incorrectly aligned objects may result in inefficient memory accesses or, on architectures with stricter alignment requirements, hardware faults. In C++, violating the alignment requirements of an object can also result in undefined behavior. Consequently, explicit control over memory alignment can be important when implementing data structures, custom allocators, memory pools, serialization mechanisms, and other low-level components.

Before C++ 11, the language provided limited standardized facilities for expressing and manipulating alignment requirements. Boost.Align addressed this limitation by providing portable alignment utilities that could be used in C++03 code and across implementations with incomplete or inconsistent support for newer alignment-related facilities.

C++11 introduced standardized support for specifying alignment requirements through language and library facilities. In particular, the language introduced alignment specifications through `alignas`, while the standard library introduced facilities such as `std::align`. However, compiler and standard library support was not always complete or consistent at the time Boost.Align was developed. The library therefore also provided portable implementations and compatibility facilities for environments in which the corresponding standard functionality was unavailable or unreliable.

6.1.2 Functionality

Boost.Align provides functionality for both dynamic memory allocation and pointer alignment. Its facilities can broadly be divided into allocation, pointer alignment, alignment inspection, and alignment diagnostics.

For dynamic memory allocation, the library provides:

- `aligned_alloc(alignment, size)`, which allocates a block of memory satisfying a specified alignment requirement;
- `aligned_free(ptr)`, which releases memory previously allocated using the corresponding aligned allocation mechanism.

These functions are useful when the default allocation provided by the runtime or standard library does not guarantee the alignment required by an application or data structure.

The library also provides allocator abstractions, including:

- `aligned_allocator<T>`.;
- `aligned_allocator_adaptor<Allocator>`.;
-

- `aligned_delete<T>`..

The allocator abstractions allow alignment requirements to be integrated with C++ allocation mechanisms and standard containers. This is particularly useful for containers whose elements must satisfy an alignment requirement greater than the default alignment provided by the allocator.

Another important part of Boost.Align concerns pointer alignment. The library provides an implementation of functionality equivalent to `std::align`, allowing a pointer within an available memory region to be adjusted to satisfy a requested alignment while preserving the required amount of available storage.

This functionality is relevant when implementing custom memory-management mechanisms. For example, an application may reserve a larger memory region and subsequently divide it into several aligned regions for different objects. Explicit pointer alignment allows such regions to be placed at addresses compatible with the alignment requirements of the objects they contain.

The library additionally provides facilities for querying and inspecting alignment requirements. These capabilities allow software to determine the alignment associated with a type or address and to verify whether an address satisfies a particular alignment constraint. Such facilities are useful when implementing generic low-level components where the alignment requirements cannot be assumed statically.

6.1.3 Relevance to automotive software

Memory alignment is particularly relevant to automotive software because many automotive systems operate under strict constraints regarding performance, memory consumption, determinism, and hardware compatibility. Software components may interact directly with hardware, communication buffers, memory-mapped data, DMA regions, or specialized data structures whose layout and alignment must be carefully controlled.

For example, an automotive application may process large collections of sensor data or communication frames using structures with specific alignment requirements. Correct alignment can contribute to efficient memory accesses and may be necessary to satisfy the requirements of a particular processor or hardware interface.

Alignment can also become important when software is designed to run across multiple processor architectures. Automotive platforms may use different CPU architectures and compiler toolchains, each with potentially different alignment characteristics. A portable abstraction for alignment can therefore reduce the amount of platform-specific code required by a software component.

Furthermore, automotive software frequently contains legacy C++ codebases that were developed before the widespread availability of modern C++ standards. Such systems may

continue to rely on Boost libraries to provide functionality that was historically unavailable or insufficiently portable in the C++ language and standard library.

Although many of the capabilities historically provided by Boost.Align are now available through modern C++ language and standard library facilities, Boost.Align remains relevant when maintaining or migrating legacy systems. Reimplementing these facilities in Rust therefore provides an opportunity to investigate how low-level memory-management functionality originally designed for C++ can be expressed using Rust's stronger type system and ownership model.

6.1.4 Challenges of reimplementation in Rust

Reimplementing Boost.Align in Rust presents several challenges. The most significant is the interaction between explicit memory alignment and Rust's safety guarantees.

Rust provides alignment information as part of its type system and guarantees that references to properly typed values satisfy the corresponding alignment requirements. However, operations involving raw pointers, manually allocated memory, and custom allocation strategies may require the use of `unsafe` code. Consequently, an implementation of Boost.Align-like functionality must carefully isolate unsafe operations and ensure that the safety invariants expected by the Rust type system are preserved.

Dynamic aligned allocation is another important consideration. Rust's allocation APIs allow an allocation to be described through a `Layout`, which contains both the size and alignment of the requested memory region. This provides a natural abstraction for implementing functionality similar to Boost.Align's aligned allocation mechanisms. Nevertheless, the implementation must correctly handle invalid layouts, allocation failures, deallocation, and the relationship between an allocation's size and alignment.

Pointer alignment presents an additional challenge because it requires reasoning about raw memory addresses and available storage. A Rust implementation must ensure that pointer arithmetic does not violate the language's safety requirements and that any resulting pointer is only converted into a reference when all required invariants are satisfied.

Allocator integration also differs significantly between C++ and Rust. Boost.Align integrates with the C++ allocator model and standard containers, whereas Rust uses the `GlobalAlloc` and allocator-related abstractions available in the language ecosystem. Therefore, a direct translation of the C++ interfaces would not necessarily result in an idiomatic Rust implementation. Instead, the reimplementation should preserve the observable semantics of the original library while adapting its interface to Rust's ownership, borrowing, and allocation abstractions.

Finally, portability must be considered. The original Boost library was designed to compensate for differences between compilers and standard library implementations. A Rust implementation should therefore be evaluated across the relevant target architectures and

toolchains to determine whether its behavior is consistent and whether platform-specific assumptions have been introduced.

6.1.5 Rationale for selection

Boost.Align was selected because it represents a low-level library whose functionality is closely related to memory management and processor architecture. It therefore provides a useful case study for evaluating the migration of C++ systems-level functionality to Rust.

The library is also particularly suitable for examining the boundary between safe and unsafe Rust. While many high-level uses of aligned data can be expressed safely through Rust's type system, explicit allocation and pointer manipulation require lower-level primitives. This makes Boost.Align useful for assessing whether Rust can provide a safer abstraction around functionality that traditionally requires careful manual reasoning in C++.

In addition, Boost.Align provides a relatively compact and well-defined API while still containing several distinct concepts, including alignment calculation, pointer manipulation, dynamic allocation, and allocator integration. This provides sufficient implementation complexity to evaluate the migration methodology without making the selected library unnecessarily large.

Although alignment support has substantially improved in modern C++ and much of the functionality provided by Boost.Align is now represented by standard C++ facilities, the library remains relevant when dealing with older C++ codebases and portability requirements. Its reimplementation in Rust can therefore demonstrate how existing low-level C++ infrastructure can be reproduced while taking advantage of Rust's memory-safety guarantees and modern abstractions.

The original Boost.Align documentation and source code were used as the primary references for the analysis and subsequent reimplementation:

- `https://github.com/boostorg/align/blob/boost-1.91.0/doc/align.qbk`
- `https://www.boost.org/library/latest/align/`
- `https://www.boost.org/doc/libs/latest/doc/html/align.html`

6.2 Boost.Endian

6.2.1 Overview

The `Boost.Endian` library provides facilities for manipulating the byte order, or endianness, of integer and user-defined types. Endianness defines the order in which the individual bytes of a multi-byte value are represented in the memory. The two predominant byte-ordering schemes are little-endian, where the least significant byte is stored first, and big endian, where the most significant byte is stored first.

Endianness is generally transparent to application level software when the data remains within a single system. However, it becomes relevant when binary data is exchanged between systems, stored in files, transmitted over networks, or interpreted by software running on different processor architectures. In such cases, the representation of a value in memory may differ from the representation expected by the receiving system.

As example, considering a 16-bit value such as `0x0102` is represented as the byte sequence `0x01 0x02` on a big-endian system and as `0x02 0x01` on a little-endian system. If the byte representation is interpreted without considering its endianness, the received application may obtain a different numerical value, leading to incorrect behavior or data corruption. Consequently, explicitly defining and handling byte order is an important requirement for portable binary data formats and communication protocols.

Boost.Endian was developed to provide a portable and consistent abstraction for these operations. The library supports binary data portability independently of the native endianness of the host platform and provides a common interface across different C++ environments. It also addresses a secondary use case involving data size optimization by providing integer representations with sizes that are not directly available as standard C++ integer types, such as 24-bit, 40-bit and 48-bit integers. These types can be useful when implementing compact binary data formats or communication protocols.

The library is header only and in its current version, requires C++11. It can make use of compiler provided byte swapping intrinsics when available, allowing implementations to take advantage of architecture specific instructions while retaining a portable interface.

6.2.2 Functionality

Boost.Endian provides three complementary approaches for working with endian dependent data: endian conversion functions, endian buffer types and endian arithmetic types. Each approach is designed for a different balance between explicitness, ease of use, performance and control over when conversions take place.

Endian conversion functions

The conversion function approach uses the standard C++ integer types to store values and explicitly converts their representation when required. The library provides both mutating and non-mutating conversion operations, including unconditional and conditional variants.

For example, an application can convert a value from big endian to the native byte order before performing calculations and convert it back before writing the value to an external representation. This approach makes the conversion points explicit and is particularly appropriate when an application already has an established data model based on the native C++ types.

The conversion functions also provide a relatively direct abstraction over underlying processor byte swapping operations. When compiler intrinsics are available, Boost.Endian can use them to generate efficient machine code.

Endian buffer types

Endian buffer types store values in a representation with a specified byte order. Unlike ordinary native integer types, the representation maintained by the buffer does not change depending on the host architecture.

The library provides buffer types for a range of integer sizes, including 8-bit, 16-bit, 24-bit, 32-bit, 40-bit, 48-bit, 56-bit and 64-bit integers. Both aligned and unaligned representations are available, with unaligned representations being particularly useful when dealing with tightly packed binary formats.

An important characteristic of buffer types is that the conversions are explicit. A value must be explicitly loaded from or stored into the endian representation. This prevents hidden conversions and gives the programmer precise control over when byte order transformation occurs.

This property is useful for binary protocols and file formats where the in memory representation must remain stable regardless of the architecture on which the software is executed.

Endian arithmetic types

Endian arithmetic types combine a fixed byte-order representation with an interface similar to the built-in C++ arithmetic types. They support arithmetic operations while automatically performing the required conversions between the stored representation and the native representation.

This approach simplifies application code because explicit conversions do not have to be performed for every arithmetic operation. For example, an application can maintain a structure containing big-endian integer fields and operate on those fields using normal arithmetic expressions.

The convenience of implicit conversion comes with a potential performance trade-off. Repeated operations on endian arithmetic types may result in repeated conversions, particularly inside computationally intensive loops. Boost.Endian therefore also documents patterns for moving conversions outside performance-critical loops when necessary.

6.2.3 Binary data portability

One of the primary motivations for Boost.Endian is binary data portability. Binary formats frequently specify the exact byte ordering and size of their fields. Unlike textual representations, binary formats cannot rely on the receiving system to interpret values according to its native representation.

This is particularly important for network protocols, file formats, serialization systems, and inter-process communication mechanisms. A data structure containing several integer fields may have to be transmitted from a little-endian processor to another system whose native representation is different. Explicitly representing the required byte order ensures that the same logical value is reconstructed at the destination.

Boost.Endian therefore separates the representation of externally defined binary data from the native representation of the host machine. This allows the application to use a consistent representation for data exchanged with external systems while remaining independent of the processor's native byte order.

The library also supports unaligned integer representations. This is useful for binary formats where fields are packed without the padding that would normally be introduced by C++ object alignment. Boost.Endian specifically identifies unaligned 24-, 40-, 48-, and 56-bit representations as useful when the exact size of the external representation matters.

6.2.4 Relevance to automotive software

Endianness is particularly relevant to automotive software because modern automotive systems consist of heterogeneous hardware and software components that communicate using precisely defined binary data formats.

Automotive systems commonly exchange data between electronic control units, sensors, gateways, communication interfaces, and higher-level computing platforms. These components may use different processors, operating systems, compilers, and software stacks. Consequently, communication interfaces cannot generally rely on the native representation of a particular processor.

The problem becomes particularly relevant when software processes binary communication frames or serialized data structures. A protocol may define a field as a 16-bit or 32-bit integer with a specific byte ordering, independently of the architecture executing the software. The implementation must therefore interpret the field according to the protocol definition rather than according to the host architecture.

This characteristic makes endian handling relevant to automotive communication stacks, serialization libraries, diagnostic protocols, and other components operating close to hardware and communication interfaces.

The relevance of these concerns is also reflected in modern automotive software platforms. The Eclipse S-CORE project, for example, is an open-source initiative focused on a software-defined vehicle platform and provides an example of the increasing role of modern software architectures and cross-platform technologies in automotive systems.

For an automotive software architecture, using explicit endian-aware representations can therefore help establish a clear boundary between the internal representation of application data and the representation required by an external communication or storage format.

6.2.5 Memory representation and portability

Boost.Endian is also relevant to portability because it avoids assuming that the host architecture uses the same byte order as the external data source.

A naïve implementation may interpret a binary structure directly through a native C++ structure. Such an approach can introduce portability problems because the resulting representation can depend on the processor's endianness, alignment requirements, padding rules, and compiler behavior.

For example, a structure containing several native integer fields cannot necessarily be treated as a portable representation of a network or file format simply by writing its memory contents directly to a file or communication channel. The representation of the fields may differ between platforms, and additional padding may be introduced by the compiler.

Endian-aware types provide a mechanism for explicitly representing the byte order required by the external format. This allows the application to distinguish between the representation used for communication and the representation used internally for computation.

This distinction is particularly valuable in safety- and reliability-oriented systems because it makes assumptions about binary representation explicit rather than relying on implicit properties of the target architecture.

6.2.6 Performance considerations

Byte-order conversion is generally a low-level operation that can be implemented efficiently using processor instructions. Boost.Endian takes advantage of compiler intrinsics when available, including byte-swap operations provided by GCC, Clang, and Microsoft Visual C++.

The performance characteristics of the different Boost.Endian approaches depend on how frequently conversions occur. When conversion functions are used, the programmer can explicitly control when a value is converted. This can be advantageous when a value is used repeatedly because the conversion can be performed once before a computationally intensive operation and the result converted back only when necessary.

Endian arithmetic types provide greater convenience but may perform implicit conversions during arithmetic operations. Boost.Endian's documentation demonstrates that, particularly in repeated computations, explicitly controlling the location of conversions can avoid unnecessary byte swapping.

This distinction is relevant for automotive applications because software may simultaneously have strict requirements for portability and computational efficiency. Communication-oriented

code may prioritize explicit representation and correctness, while performance-critical processing may require minimizing repeated conversions.

6.2.7 Challenges of reimplementation in Rust

Reimplementing Boost.Endian in Rust presents several challenges, although Rust provides several primitives that are well suited to the problem.

The first challenge is representing values with an explicitly defined byte order while maintaining Rust's type and memory-safety guarantees. Rust's standard library provides methods such as `from_be`, `from_le`, `to_be`, and `to_le`, as well as byte-oriented conversion facilities. These provide a strong foundation for implementing endian-aware abstractions, but they do not directly reproduce the complete Boost.Endian API.

A second challenge is the implementation of endian-aware buffer types. Such types need to preserve a specific byte representation independently of the host architecture. A Rust implementation can use byte arrays as the underlying representation, which has the advantage of making the representation explicit and avoiding direct dependence on native integer layout.

Unaligned representations introduce another challenge. Rust provides strong guarantees around references and alignment, meaning that interpreting an arbitrary byte sequence directly as an integer reference may require unsafe operations. A safe implementation should therefore prefer byte-based representations and explicit conversion operations where possible, isolating any required unsafe code behind a well-defined abstraction.

A further challenge is reproducing Boost.Endian's arithmetic types. Boost.Endian allows endian-aware values to participate directly in arithmetic expressions, while Rust's operator traits provide a different abstraction mechanism. Implementing equivalent behavior requires implementing traits such as `Add`, `Sub`, `Mul`, and their assignment variants where appropriate.

However, a direct translation of the C++ API would not necessarily produce an idiomatic Rust library. Rust's ownership model, trait system, explicit conversion mechanisms, and emphasis on compile-time safety provide opportunities to design a safer interface rather than reproducing the C++ interface exactly.

Another important consideration is floating-point support. Endianness of floating-point representations cannot always be treated as a simple integer byte swap because floating-point representation and byte ordering involve additional assumptions. Boost.Endian therefore places restrictions on some floating-point conversion operations. A Rust reimplementation must preserve these semantic limitations rather than assuming that arbitrary floating-point values can safely be transformed by reversing their bytes.

6.2.8 Rationale for selection

Boost.Endian was selected because it represents a particularly relevant example of low-level, cross-platform functionality that is commonly required when implementing systems software.

Unlike functionality that is primarily concerned with convenience abstractions, endian handling directly affects the representation of data exchanged between software components. This makes the library particularly relevant to automotive architectures, where multiple software and hardware components must exchange binary data according to well-defined protocols.

The library also provides an interesting migration target because it contains several distinct implementation strategies: conversion functions, buffer types, and arithmetic types. These approaches expose different trade-offs between explicitness, ease of use, memory representation, and performance. Reimplementing them in Rust provides an opportunity to evaluate how these design choices can be expressed using Rust's type system and trait-based abstractions.

The support for both aligned and unaligned representations is another reason for its selection. It introduces a direct connection to low-level memory representation and requires careful consideration of alignment, raw byte access, and safe abstractions. This complements the selection of `Boost.Align`, which focuses explicitly on memory alignment.

The combination of `Boost.Align` and `Boost.Endian` is therefore particularly useful for the migration study. While Boost.Align focuses on the physical placement and alignment of objects in memory, Boost.Endian focuses on the representation and interpretation of multi-byte values. Together, they cover two closely related aspects of low-level systems programming: how data is positioned in memory and how that data is represented across system boundaries.

Furthermore, Boost.Endian is sufficiently mature and well-defined to provide a meaningful basis for comparing the original C++ implementation with a Rust implementation. The library's documented performance considerations, multiple API approaches, and portability requirements provide several dimensions against which the Rust implementation can be evaluated.

The primary references used for the analysis are:

- `https://www.boost.org/library/latest/endian/`
- `https://www.boost.org/doc/libs/latest/libs/endian/doc/html/endian.html`
- `https://github.com/eclipse-score`

Chapter 7

Implementation of the Migration Strategy

7.1 ImplementationOverview

This chapter presents the implementation of the Boost to Rust migration described in the previous chapters. The implementation focuses on two selected Boost libraries, namely `Boost.Align` and `Boost.Endian`, which were selected due to their low level nature, relevance to systems programming and suitable for evaluating the migration methodology.

The implementation translates the relevant functionality of the original C++ libraries into Rust, while avoiding a direct syntactic translation of the original source code. Instead, the Rust implementation uses Rust's ownership model, type system and stadard library facilities where these provide suitable abstractions for the original C++ functionality.

The implementation is evaluated with respect to foir main objectives: functional equivalence, memory safety, interoperability with existing C++ software and suitability for automotive systems software.

The implementation follows the migration strategy described in Chapter 5. For each library, the original functionality is first mapped to Rust abstractions, follower by the implementation of the selected functionality and its associated tests. Where interoperability is required, a C++ interface is introduced around the Rust implementation, which is then tested through benchmark tests.

7.2 Implementation Environment

The implementation was developed in the `boostinrust` repository, which provides the experimental environment for evaluating the migration of selected Boost libraries to Rust. The implementation focuses on `Boost.Align` and `Boost.Endian`, selected as representative examples of low level systems functionality involving memory alignment and binary data representation.

The Rust implementation was developed using Rust version 1.93. The development environment includes the Rust compiler (`rustc`), Cargo and the LLVM/Clang based tooling provided with the Rust toolchain. In particular, the environment contains the Rust compiler and the associated `rust-clang`, `rust-lld`, `rust-llvm-dwp`, and `rustdoc` components. The use of the LLVM/Clang toolchain is relevant to the low-level nature of the selected libraries and provides the compiler infrastructure required for building and inspecting the resulting Rust implementation.

The original Boost implementations are used as the reference implementations for the migration. This allows the Rust implementation to be evaluated against the functionality and observable behavior of the original C++ corresponding libraries. The approach follows the migration strategy described in Chapter 5, in which the functionality and dependencies of the original library are first analyzed before equivalent Rust abstractions are defined and implemented. The resulting Rust implementation is then tested and evaluated against the original C++ library to ensure functional equivalence, memory safety, interoperability and suitability for automotive systems software.

The implementation does not attempt to reproduce the interal structure of the C++ implementation directly. Instead, Rust specific mechanisms are used where they provide a more appropriate representation of the original functionality. This includes Rust's ownership and borrowing model, type system and standard library facilities for memory layout and byte order operations. This distinction is particularly relevant for the selected libraries. Boost.Align requires operations involving memory alignment, allocation and pointer manipulation, while Boost.Endian requires explicit control over the byte representation and byte order of stored values.

The implementation is structured to support functional verification through tests. The behavior of the Rust implementation is evaluated against the expected behavior of the corresponding Boost functionality, with particular attention given to boundary conditions and low-level memory operations. This follows the development plan established in Chapter 4, where functional equivalence is verified through unit testing and interface-level validation.

A specific concern of the implementation environment is the use of `unsafe` Rust. Although Rust provides alignment and memory-safety guarantees through its type system, reproducing some low-level Boost functionality requires operations involving raw pointers or manually allocated memory. uch operations are therefore isolated and minimized in the implementation. This follows the migration strategy's requirement to preserve Rust's safety guarantees while restricting unsafe operations to well-defined boundaries.

Table 7.1 summarizes the main tools and versions used during the implementation.

Component	Version / Configuration
Rust	1.93
Rust compiler	`rustc 1.93`
Documentation	`rustdoc 1.93`
Compiler infrastructure	LLVM / Clang
Linker	`rust-lld`
Reference implementation	Boost 1.91.0

TABLE 7.1: Implementation environment

The resulting environment provides a controlled basis for implementing and evaluating the selected libraries. It combines the original Boost implementations used as behavioral references with a contemporary Rust toolchain, allowing the migration to be evaluated while preserving the low-level characteristics relevant to systems programming.

7.3 Boost.Align Reimplementation

The reimplementation of Boost.Align focuses on the memory-alignment functionality required by the selected use case. Rather than reproducing the complete Boost.Align API, the implementation provides a Rust-native abstraction for creating and manipulating 32-byte-aligned data and exposes the relevant functionality through a C-compatible interface.

The implementation is located in the align component of the project. Its central abstraction is AlignedBuffer, which stores floating-point data in an alignment-aware representation suitable for SIMD processing. The implementation additionally provides an `aligned_type!` macro for defining aligned Rust types and an AVX-specific processing function for operating on aligned buffers.

This approach follows the migration strategy described in Chapter 5: the objective is not to reproduce the original C++ implementation internally, but to preserve the relevant observable behaviour while using Rust's type system, ownership model and memory-management mechanisms. In particular, the implementation uses Rust's `#[repr(align(...))]` attribute and Vec allocation rather than implementing a separate low-level aligned allocator.

7.3.1 Scope and API mapping

The Boost.Align library provides several facilities related to memory alignment, including aligned allocation and deallocation, pointer alignment, alignment inspection and aligned allocators. For the implementation developed in this project, only a subset of this functionality was required.

The Rust implementation therefore maps the relevant alignment requirements to Rust language and standard-library mechanisms rather than attempting to reproduce the complete Boost.Align API.

Boost.Align concept	**Rust implementation**	**Purpose**
Aligned storage	`#[repr(align(32))]`	Establishes a compile-time alignment requirement
Aligned type	`Aligned<T>`	Provides a generic 32-byte-aligned wrapper
User-defined aligned type	`aligned_type!`	Generates types with a configurable alignment
Aligned memory buffer	`AlignedBuffer`	Stores aligned `f32` data
Aligned allocation	`Vec<F32x8>`	Uses Rust's allocator with the alignment of `F32x8`
Pointer access	`as_ptr()` / `as_mut_ptr()`	Provides raw-pointer access for SIMD and FFI
Alignment-dependent processing	`double_f32_avx()`	Performs AVX operations on aligned memory
C++ interoperability	`ffi` module	Exposes the buffer and SIMD operations through C ABI functions

TABLE 7.2: Mapping of Boost.Align concepts to the Rust implementation

This represents a deliberate reduction of the original API surface. In particular, the current implementation does not provide direct Rust equivalents of all Boost.Align functions such as align_up, align_down, is_aligned, aligned_alloc, aligned_free, or aligned_allocator. Instead, alignment is established structurally through the type definition of the stored elements.

This distinction is important because the objective of the implementation is to investigate whether the relevant alignment guarantees can be expressed more directly using Rust's type system rather than reproducing Boost's low-level allocation interface.

Bibliography

70% of Security Bugs Are Memory Safety Issues (2020). Tech. rep. Microsoft Security Response Center.

Amazon Web Services (2026). *Firecracker: Secure and Fast MicroVMs for Serverless Computing*. Firecracker main page. URL: https://firecracker-microvm.github.io/.

AUTOSAR Partnership (2023). *AUTOSAR Adaptive Platform - Explanation of Adaptive Platform Design*. Tech. rep. R23-11. Describes the service-oriented C++ architecture used in modern SDVs. AUTOSAR GbR. URL: https://www.autosar.org/standards/adaptive-platform.

Blandy, Jim, Jason Orendorff, and Leonora F.S. Tindall (2021). *Programming Rust: Fast, Safe Systems Development*. Sebastopol, CA: O'Reilly Media. ISBN: 978-1-492-05259-3. URL: https://oreilly.com.

Boost C++ Libraries Documentation (2025). URL: https://www.boost.org/docs/.

Bristol, Andrea (2025). *Ada and Rust are highlighted by the NSA and CISA in Memory Safe Language Information Sheet*. URL: https://www.adacore.com/blog/ada-and-rust-are-highlighted-by-the-nsa-and-cisa-in-memory-safe-language-information-sheet.

C++ ABI Compatibility (2025). Accessed: 2025-01-18. URL: https://parallel-rust-cpp.github.io/cpp_abi.html.

CppCon (2020). *CppCon 2020 Presentation Materials*. Cpp Conference. URL: https://github.com/CppCon/CppCon2020.

crates.io: The Rust Community's Crate Registry (2025). URL: https://crates.io/crates?sort=recent-downloads.

Dashdamirli, N. (2025). *Analyzing the Performance and Practicality of C, Rust, Python, and Lua Programming Languages for Developing Microcontroller Applications*. Technical report.

Deloitte (2024). *Software-defined vehicles: Global manufacturer readiness study*. URL: https://www.deloitte.com/ua/en/Industries/automotive/analysis/software-defined-vehicles.html.

Foundation, Eclipse (2025). *Eclipse Safe Open Vehicle Core (S-Core)*. URL: https://projects.eclipse.org/projects/automotive.score.

Google (2021). *High-level design of C++/Rust interop*. States impossibility of using C++ templates via standard bindgen/C FFI. URL: https://crubit.rs/design/design.html.

Google Android Team (2024). *Rust / C++ Interop in Android Platform*. Case study of large-scale hybrid C++/Rust integration in a production OS. URL: https://source.android.com/docs/setup/build/rust/building-rust-modules/cpp-interop.

Hong, J. (2023). “Improving Automatic C-to-Rust Translation with Static Analysis”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*.

Immunant, Inc. (2025). *c2rust: Migrate C code to Rust*. Version 0.21.0. URL: https://github.com/immunant/c2rust.

ISO (2018). *ISO 26262:2018 Road vehicles – Functional safety*. Part 6: Product development at the software level. Geneva, Switzerland: International Organization for Standardization.

ISO 26262-6:2018 Road vehicles — Functional safety — Part 6: Product development at the software level (2018). Defines requirements for software unit testing and freedom from interference. Geneva, Switzerland: International Organization for Standardization. URL: https://www.iso.org/standard/68388.html.

ISO/IEC 14882:2020 Programming languages — C++ (Dec. 2020). Defines the C++20 standard, including modules, coroutines, concepts, and ranges. Geneva, Switzerland: International Organization for Standardization. URL: https://www.iso.org/standard/79358.html.

Khan, Mustafif (2023). *Rust for C++ Programmers: Learn how to embed Rust in C/C++ with ease*. London, UK: BPB Online. ISBN: 978-93-5551-355-7. URL: https://www.bpbonline.com.

Klabnik, Steve, Carol Nichols, and The Rust Project Developers (2017). *The Rust Programming Language*. San Francisco, CA: No Starch Press. ISBN: 978-1593278281. URL: https://doc.rust-lang.org/book/.

Kuo, Akhil (2024). *Rust for Blockchain Application Development: Use the Power of Rust to Develop Scalable, Robust, and Secure Blockchain Applications*. Birmingham, UK: Packt Publishing. ISBN: 978-1-83763-464-4. URL: https://www.packtpub.com/en-us/product/rust-for-blockchain-application-development-9781837634644.

Ling, M. et al. (2022). “In Rust We Trust: A Transpiler from Unsafe C to Safer Rust”. In: *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. Pittsburgh, PA, USA, pp. 354–355.

Memory Safety in Android and Chrome (2022). Tech. rep. Google Project Zero.

Meneely, A. et al. (2025). “Just Use Rust: A Best-Case Historical Study of Open Source Vulnerabilities in C”. In: *2025 IEEE/ACM 3rd International Workshop on Software Vulnerability Management (SVM)*.

Mozilla (2026). *Including Rust Code in Firefox*. Firefox Source Docs documentation. Mozilla. URL: https://firefox-source-docs.mozilla.org/build/buildsystem/rust.html.

Okawa, S. and S. Yamaguchi (2024). “A Performance Study on Rust and C Programs”. In: *2024 Twelfth International Symposium on Computing and Networking (CANDAR)*.

Rooney, M. P. and S. J. Matthews (2023). “Evaluating FFT Performance of the C and Rust Languages on Raspberry Pi Platforms”. In: *2023 IEEE International Conference on Cluster Computing (CLUSTER)*.

Rust Documentation (2025). Accessed: 2025-01-18. URL: https://docs.rs/.

S&P Global Mobility (2025). *The software-defined vehicle market is taking off*. URL: https://www.spglobal.com/automotive-insights/en/blogs/2025/09/the-software-defined-market-is-taking-off.

Stroustrup, Bjarne (2013). *The C++ Programming Language*. 4th ed. Addison-Wesley.

– (2020). “Thriving in a Crowded and Changing World: C++ 2006–2020”. In: *Proceedings of the ACM on Programming Languages* 4.HOPL, 70:1–70:168. DOI: 10.1145/3386320. URL: https://www.stroustrup.com/hopl20main-p5-p-bfc9cd4--final.pdf.

Synopsys (2025). *What is ASIL (Automotive Safety Integrity Level)?* Accessed: 2026-01-31. URL: https://www.synopsys.com/glossary/what-is-asil.html.

The Linux Kernel Organization (2026). *Rust — The Linux Kernel documentation*. Documentation for experimental Rust support in the Linux kernel (v6.1+). URL: https://docs.kernel.org/rust/index.html.

The MISRA Consortium (Oct. 2023). *MISRA C++:2023 Guidelines for the use of C++17 in critical systems*. Industry standard attempting to mitigate C++ memory safety risks via subsetting. MISRA Ltd. Nuneaton, UK. ISBN: 978-1-906400-33-3. URL: https://misra.org.uk/.

The Rust Project Developers (2026). *Embedded Devices - Rust Programming Language*. Rust for embedded devices. URL: https://www.rust-lang.org/what/embedded.

The rust-bindgen Project Developers (2026). *Crate bindgen: Generate Rust bindings for C and C++ libraries*. Rust documentation. URL: https://docs.rs/bindgen/latest/bindgen/.

Tolnay, David (2025). *cxx: Safe Interop between Rust and C++*. The industry-standard library for safe, static-analysis-driven FFI between Rust and C++. URL: https://github.com/dtolnay/cxx.

Xia, L., Y. Wu, and B. Hua (2023). “Rustcheck: Safety Enhancement of Unsafe Rust via Dynamic Program Analysis”. In: *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*.

Zakarum (2023). *boosters: Rust port of boost libraries for C++*. Version 0.1.0. Rust crate version 0.1.0. URL: https://crates.io/crates/boosters.